across cloud providers) platform that is ridiculously easy to deploy and use. Some core concepts on which Parseable is built are listed below.

No need for indexing: Traditionally, text search engines like Elastic, etc, have doubled up as log storage platforms. This made sense because log data had to be searched for it to be really useful. But this came at a high cost – indexing is CPU-intensive and slows down the ingestion. Additionally, most of these indexing systems generate index data in the same order of the raw log data. This doubles the storage cost and increases complexity. Parseable changes this. With columnar data formats (parquet), it is possible to compress and query the log data efficiently without indexing it.

Ownership of both data and content: With parquet as the storage format, stored on standard object storage buckets, users not only own their log data but also have unabridged access to the actual content. This means they can simply point their analysis tools like Spark, Presto or TensorFlow to extract more value out of the data. This is an extremely powerful feature, opening up new avenues of data analysis.

Fluid schema: Logs are generally semi-structured by nature, and they are ever evolving, e.g., a developer may start with a log schema like this:

```
{
  "Status": "Ready",
  "Application": "Facebook"
}
```

But as more information gets collected, the log schema may evolve to:

```
{
  "Status": "Ready",
  "Application": {
      "UserID": "asdadaferssda",
      "Name": "Facebook"
  }
}
```

Engineering and SRE teams regularly face schema related issues. Parseable solves this with a fluid schema approach that lets users change the schema on the fly.

SDK-less ingestion: The current ingestion mechanism to logging platforms is quite convoluted, with several protocols and connectors floating around. Parseable aims to make log ingestion as easy as possible. This means you can simply use HTTP POST calls to send logs to Parseable; no complicated SDKs are required. Alternatively, if you want to use a logging agent like FluentBit, Vector, LogStash, etc, almost all the major log collectors already have support for HTTP; hence, Parseable is already compatible with your favourite log collection agent.

Getting started

Let's see how to get started with Parseable now. We'll use the Docker image to try out Parseable.

We'll use Parseable with a publicly accessible object storage, just to help you experience it. Here is the command:

```
cat << EOF > parseable-env
P_S3_URL=https://minio.parseable.io:9000
P_S3_ACCESS_KEY=minioadmin
P_S3_SECRET_KEY=minioadmin
P_S3_REGION=us-east-1
P_S3_BUCKET=parseable
P_LOCAL_STORAGE=/data
P_USERNAME=parseable
P_PASSWORD=parseable
EOF

mkdir -p /tmp/data
docker run \
  -p 8000:8000 \
  --env-file parseable-env \
  -v /tmp/data:/data \
  parseable/parseable:latest
```

You can now log in to the Parseable UI using the credentials we passed here, i.e., 'parseable' , 'parseable'. This will have some data, because Parseable is pointing to the publicly open bucket.

Note that this is a public object storage bucket, which we are using for demo purposes only. Make sure to change the bucket and credentials to your object storage instance before sending any data to Parseable.

To get a deeper understanding of how Parseable works and how to ingest logs, please refer to the documentation available at *https://www.parseable.io/docs/introduction.*

I hope this article helped you get better insights into the logging ecosystem, and that you find Parseable an interesting project to try out and, hopefully, join the community. END

By: Nitish Tiwari

The author is the founder of Parseable. You can find more details at *https://github.com/nitisht.*